

Agentic Coding for Economists

Day 3: Orchestration, Autonomous Agents, Replication

Participant repo: https://github.com/meleantonio/AC4E_Pavia

Antonio Mele
University of Pavia

June 22–24, 2026



Day 2 Recap

What we built:

- Research design memo: question, literature, inputs, method, assumptions, limitations
- SDD chain: intent $\rightarrow$ requirements $\rightarrow$ design $\rightarrow$ tasks
- Literature map plus data or model-input map
- Prioritized issues with acceptance criteria and verification evidence

Context discipline: tight, wide, and external context recorded in [notes/context_patterns.md](#).

Day 3 Goals

- ❶ **Skills and review roles:** reuse procedures and separate implementation from review
- ❷ **Subagents and MCP:** specialized contexts and structured external access where they add value
- ❸ **Cloud agents and swarms:** branch-isolated work with issues, labels, handoffs, and review-before-merge
- ❹ **Autonomous-agent risk:** evaluate read/write boundaries, memory, execution, approvals, and evidence
- ❺ **Final integration:** replication README, 5–7 minute presentation, and 30-day adoption plan

Definition of Done (Day 3): reviewable package, not a chat transcript.

Why Advanced Workflows Come Later

Before adding more autonomy

Skills, subagents, MCP, cloud agents, and swarms are useful only after the repository has scope, boundaries, and review criteria.

- **Needed first:** repo, project instructions, privacy boundaries, project brief, design memo, review criteria.
- **Without those:** advanced agents only automate confusion.
- **Today:** use advanced workflows to make review sharper, not to bypass judgment.

What is MCP?

MCP = Model Context Protocol

- A standard for connecting AI models to **external tools** (APIs, databases, filesystems)
- Supported agents invoke MCP servers; the AI then “sees” tools and resources they expose
- Use case for economists: pull FRED series, read data from controlled folders, search the web — all within a single chat
- **Local statistical software**: MCP can bridge to **Stata**, **MATLAB**, or **R** on your machine (via appropriate servers), so the agent can run batch jobs, scripts, or pull outputs in your environment—with your approval

MCP = bridges to real tools. The AI gains structured access instead of hallucinating data.

MCP Configuration

Locations:

- **Cursor:** `~/.cursor/mcp.json` or `project .cursor/mcp.json`
- **Claude Code:** CLI MCP setup or `project .mcp.json`
- **Codex:** MCP configured through Codex configuration

Security: Never put API keys in `mcp.json` in git. Use `env` with `${VAR}`.

FRED API Example

```
{
  "mcpServers": {
    "mcp-fredapi": {
      "command": "uv",
      "args": [
        "--directory", "~/mcp-fredapi",
        "run", "--with", "mcp", "--with", "httpx",
        "mcp", "run", "server.py"
      ],
      "env": {
        "FRED_API_KEY": "${FRED_API_KEY}"
      }
    }
  }
}
```

Key from: <https://fredaccount.stlouisfed.org/apikeys>

When MCP Adds Value vs Overkill

Adds value:

- Recurring data pulls (FRED, ECB, BIS)
- Scoped filesystem access (e.g. data/ only)
- Web search for literature
- Local Stata/MATLAB/R workflows (MCP bridge to your machine)

Overkill:

- One-off data download (browser + save)
- Single analysis (manual run)
- Heavy custom tooling

Economists: FRED is the obvious win. Start with one MCP.

What Are Subagents?

Subagents are specialized AI assistants with their own context window.

- Each gets its own **prompt**, **tools**, and optionally a different **model**
- Use when you need: **context isolation**, **parallel work**, or **role specialization**
- Differ from main agent: focused on one job; from skills: separate context

Economics example: “Have pr-reviewer review the code changes and bibliographer check the references section” — two subagents in parallel.

Subagent / Custom Agent Format

Each subagent is a markdown file with **YAML frontmatter** and a prompt body:

Frontmatter fields

```
name: invocation (e.g. pr-reviewer)
description: when Agent delegates — be specific
model: fast or inherit
readonly: true for reviewers
```

Scope

```
Cursor: .cursor/agents/
Claude Code: .claude/agents/
Codex: explicit subagent workflow or
tool-native config
```

Description drives delegation. Vague → Agent rarely uses it.

Verify field names and UI labels in your tool version.

Subagent Example: PR Reviewer

```
---  
name: pr-reviewer  
description: PR review specialist for econometric code.  
  Use when reviewing pull requests for consistency,  
  statistical correctness, and style.  
model: fast  
readonly: true  
---
```

You are a rigorous code reviewer for econometric projects.

When invoked on a PR:

1. **Consistency**: Project rules, naming, patterns
2. **Correctness**: Coefficients, SEs, missingness, units
3. **Style**: Docstrings, citations, README

Report: pass / needs changes + line-level suggestions.

Role Specialization: Reviewer, Replicator, Bibliographer

pr-reviewer Consistency, correctness, style on PRs. Check coefficient interpretations, unit labels, citation format.

replicator Verify pipeline runs from clean state; no hardcoded paths; reproducible output hash.

bibliographer Standardize .bib entries, fix journal abbreviations, cross-check in-text citations.

One clear role per subagent. Avoid generic “helper” agents.

When to Use Subagents vs Main Agent vs Skills

Use subagent	Multi-step review, long exploration, parallel streams
Use skill	Single-purpose quick task (e.g. generate changelog)
Use main agent	General orchestration, decisions, merging

Invoke explicitly: “Use the pr-reviewer subagent to review PR #42” or /pr-reviewer review this PR

What Are Skills?

Skills extend the agent with reusable workflows.

- Loaded when the **description** matches the user request
- Unlike subagents: **no separate context** — they augment the main agent
- Same pattern across tools: a discoverable SKILL.md with optional references/, scripts/, and assets/

Example trigger: “Verify that my pipeline in scripts/run_analysis.py is reproducible” → replication-checker skill loads

Anatomy:

- **Frontmatter:** name, description (required). Description is the primary trigger.
- **Body:** instructions, workflows, when to read references/ or scripts/
- **Optional:** references/, scripts/, assets/ for progressive disclosure

Progressive disclosure: Keep SKILL.md under ~500 lines. Move detailed checklists to references/.

Skill-Creator Principles

- ❶ **Concise is key:** Context window is shared. Only add what the agent doesn't already have.
- ❷ **Degrees of freedom:** Low (scripts, strict steps) for fragile ops; high (text guidance) for variable decisions.
- ❸ **Progressive disclosure:** SKILL.md stays lean; load references/ or scripts/ only when needed.

Validate-iterate: Use the skill on a real task. Fix what breaks.

Building the Replication-Checker Skill

```
---
name: replication-checker
description: Verifies that research pipelines run from clean
             state with no hardcoded paths, declared dependencies,
             and reproducible output.
---

# Replication Checker

When invoked:
- Identify main entry point (script, notebook, Make)
- Check for hardcoded paths (see references/)
- Verify dependencies: requirements.txt
- Run minimal/dry-run; report success or failure

Output: green/yellow/red, blockers, run command
```

Skill Taxonomy

Skill	Trigger phrases	Degree of freedom
replication-checker	verify replication, check pipeline	Medium
literature-mapper	standardize bib, map references	Medium
data-contract-enforcer	validate data, check units	Low (script-heavy)
paper-polisher	polish paper, citation hygiene	High

Tool paths differ; anatomy transfers: Cursor uses `.cursor/skills/`, Claude Code uses `.claude/skills/`, Codex uses `.agents/skills/`.

What Are Cloud / Remote Agents?

Cloud/remote agents run outside the local editor and return branch or PR-level work.

- They clone your repo, work on a branch, and may produce a PR.
- They are useful for long, isolated, reviewable tasks that do not need private data.
- They are a bad fit for live demos, vague research decisions, or confidential files.
- Cursor, Codex, Claude, and other tools expose this pattern differently; verify current UI and field names in official docs.

Handoff flow: Task → agent runs remotely → branch/PR created → **you review, then merge**

Branch Handoff Protocol

- ❶ **Before launch:** Create or specify the branch; ensure it's based on current main
- ❷ **Task description:** Clear scope, file paths, "work on branch X"
- ❸ **During run:** Agent clones, checks out branch, executes. You keep working.
- ❹ **On completion:** Agent pushes branch, opens PR
- ❺ **Review:** Open PR, read diff, run tests. Use pr-reviewer subagent.
- ❻ **Decision:** Merge, request changes, or close

Safety and Prompt Injection Awareness

Safety rules:

- **No merge without review** — agent output can be wrong or off-scope
- **Branch isolation** — work stays on branch until you merge
- **No secrets in prompts** — use env vars; exclude `.env` through privacy boundaries
- **Sanitize external input** — avoid piping raw user text into agent prompts

Prompt injection: Malicious content in inputs that steers the agent. Lower risk when you control inputs (your code).

Review-Before-Merge Protocol

Checklist before merging a remote-agent PR:

- Run tests
- Spot-check 3 random files
- Verify no logic changes unless requested
- No hardcoded paths or secrets in diff

Key prompt: “Generate docstrings for all Python files in `scripts/`. Use Google-style. Work on branch `agent/docstrings`. Do not modify function bodies.”

Autonomous Agents: OpenClaw, Hermes, Eve

Agent	What to notice	Workshop stance
OpenClaw	Local-first assistant; channels, tools, skills, sandboxing, multi-agent routing	Good for personal-agent and channel-safety discussion; optional reading
Hermes	Nous Research agent focused on long-running context and user-specific capability growth	Useful for memory, persistence, and ownership of research context
Eve	Vercel framework for building agents with instructions, skills, tools, sandboxes, channels, subagents, schedules	Use only after checking current docs and access boundaries

Rule: More autonomy means more need for logs, review gates, and reproducibility evidence.

Autonomous-Agent Risk Card

Fill one risk card before letting any autonomous agent touch a research repo:

- **Read boundary:** what files, channels, or data can it inspect?
- **Write boundary:** what can it edit, trigger, schedule, or publish?
- **Memory:** what state persists across sessions, and who controls it?
- **Execution:** local, cloud, sandbox, or production system?
- **Review gate:** what human approval is required before outputs enter the repo?
- **Evidence:** what logs, diffs, commands, or replication outputs prove what happened?

Artifact: `agent-harness/autonomous_agent_risk_card.md`

Risk Card Decision

Approve only if you can answer:

- **Read:** what can it inspect?
- **Write:** what can it edit, trigger, publish, or schedule?
- **Memory:** what persists and who controls it?
- **Execution:** local, sandbox, cloud, production, or mixed?
- **Evidence:** which logs, diffs, transcripts, or outputs prove what happened?

Decision labels

- Safe for read-only use.
- Safe for branch-isolated write use.
- Not safe for this workshop repo.
- Needs more documentation before use.

Rule: do not connect private data unless every boundary and approval gate is acceptable.

Map New Agents Back to the Harness

Autonomous-agent feature	Course harness equivalent
Markdown instructions	AGENTS.md, CLAUDE.md, Cursor rules
Skills / tools	SKILL.md, MCP tools, validation scripts
Channels / schedules	GitHub issues, CI, scheduled checks
Subagents / workers	Reviewer, replicator, bibliographer roles
Sandbox	Branch, worktree, remote runner, protected main
Logs	PR description, orchestration log, replication README

Teaching point: The architecture transfers; the UI changes.

What Is a Swarm?

A **swarm** is **several agents coordinated** toward one project outcome — not a single prescribed toolchain.

- You might combine **foreground agent**, **subagents**, **skills**, and **remote agents** in one workflow
- Orchestration can be **human-led**, **scripted**, or **agent-assisted** (planner creates issues; implementers execute)
- The next slides use **GitHub + remote agents** as one practical pattern; adapt labels and tools to your lab

GitHub as the Control Plane

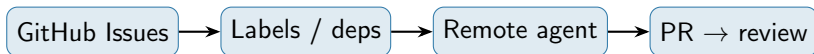
Issues = units of work. Make dependencies explicit:

- **Labels** (examples): `parallel`, `sequential`, `blocked-by:#12`, `ready`
- **Project columns** or milestones for waves: Wave A (parallel) → Wave B (after merge of #5)
- Planner (you or an agent) opens issues with **acceptance criteria**; sub-orchestrators triage and assign remote-agent runs

Rule of thumb: Independent tasks → parallel agents; dependent tasks → sequential launches after prior PR merges.

From Issues to PRs

As in the remote-agent section above: each issue can drive one **branch** and one agent run when the task is large enough to isolate.



Foreground agent on your machine is still valid for short edits; remote agents help when runs are long or you want many branches at once.

Layered Orchestration

Three layers (names are illustrative — tune to your repo):

Planner Creates or refines the issue graph (human or main Agent). Outputs: labeled issues, milestones.

Implementation swarm Remote agents (and humans) work issues in parallel or sequence per labels.

Review / consistency swarm Subagents + skills: pr-reviewer, replicator, replication-checker; optional “review lead” Agent pass across PRs

Sub-orchestrators can be **different chats** or **different remote-agent tasks** with narrow prompts (e.g. “only consistency of notation between model/ and paper/”).

Handoffs and Ledger (Still Essential)

Branches: e.g. feat/hank-ss, feat/hank-irf — one coherent branch per issue when using remote agents.

- Merge only after review; keep main stable

Handoff files: notes/handoff_*.md — column names, units, SS outputs IRFs consume

Orchestration log: notes/orchestration_log.md — Stream | Branch | Issue | Status | PR
| Reviewer

Example: HANK + Monetary Policy (Issues)

Parallel (no ordering across these): literature & BibTeX hygiene; test harness / CI stub; data download + cleaning skeleton for macro time series.

Sequential: steady-state / calibration module → shock definitions & IRF computation → aggregation & welfare table → paper section tying results to policy question.

Issue (sketch)	Label	Remote agent?
#1 SS solver + calibration	sequential / wave 1	Yes
#2 Monetary shock IRFs	blocked-by:#1	Yes, after #1 merges
#3 Heterogeneity aggregation	blocked-by:#2	Often sequential
#4 Lit review + bib	parallel	Yes or foreground
#5 Figures + policy section	blocked-by:#3	Yes

Example: Branches and Commands

Setup (illustrative):

```
git checkout main && git pull
git checkout -b feat/hank-ss
# ... launch remote agent on issue #1 for this branch; open PR when done.
```

After #1 merges: create feat/hank-irf from updated main; launch agent for issue #2.

Optional further reading (other orchestration metaphors):

<https://github.com/steveyegge/gastown>

Safety Rules: No Merge Without Review

Before merging:

- ❶ Run **pr-reviewer subagent** on each PR
- ❷ Run **replication-checker skill** on the combined pipeline
- ❸ Merge in dependency order (e.g. data → equilibrium → IRFs → paper)

Swarm lab protocol: Phase 1 Setup (15 min) → Phase 2 Parallel (90 min) → Phase 3 Review & Merge (45 min) → Phase 4 Verification (15 min)

Definition of Done for Day 3

- ➊ **Review role used:** subagent, custom agent, or saved role prompt reviews a real diff or memo.
- ➋ **Reusable workflow used:** skill-style replication checker, literature checker, or model-assumption checker.
- ➌ **Orchestration log:** [notes/orchestration_log.md](#) records delegated streams and human review.
- ➍ **Risk card:** [agent-harness/autonomous_agent_risk_card.md](#).
- ➎ **Replication path:** [replication/README.md](#) plus clean-run evidence or documented blocker.
- ➏ **Communication:** 5–7 minute presentation outline and 30-day adoption plan.

Day 3 Deliverables

Artifact	Location
Subagent(s)	pr-reviewer in your tool path
Skill(s)	replication-checker/SKILL.md
Orchestration log	notes/orchestration_log.md
Risk card	agent-harness/autonomous_agent_risk_card.md
Handoff files	notes/handoff_*.md
Replication README	replication/README.md
Presentation and plan	slides/notes plus notes/30_day_plan.md

Final Integration Before Presentation

End today by making the project coherent:

- **Paths:** scripts, docs, outputs, and README agree.
- **Definitions:** variable names, units, primitives, parameters, and outputs are consistent.
- **Evidence:** output files exist or blockers are documented.
- **Claims:** presentation language matches the evidence and limitations.
- **Privacy:** boundaries are still respected.

Goal: a coherent, reviewable research package.

Replication README

A colleague should be able to follow it without guessing.

```
# Replication README

## Overview
## Software and versions
## Data or model inputs
## Setup
## Run instructions
## Expected outputs
## Troubleshooting
## Known limitations
## Contact or ownership
```

Use your lane: Rscript, stata-mp -b do, matlab -batch, python, julia -project, or the shortest honest manual sequence.

5–7 minutes, not a full seminar:

- ➊ Research question
- ➋ Why it matters
- ➌ Data or model and method
- ➍ Main result, prototype, or blocker
- ➎ Agentic workflow: what the agent did and what you reviewed
- ➏ Replication status
- ➐ Next 30 days

Avoid: “The AI proved the effect.” Designs, assumptions, data, and review evidence support claims.

30-Day Adoption Plan

Week	Sustainable habit
1	Add or improve AGENTS.md; document privacy boundaries; define one verification check.
2	Choose one real task; create an issue; work on a branch; review the diff before merge.
3	Create one reusable skill or saved prompt and one reviewer role prompt; use them on a real artifact.
4	Run one replication check; update the README; share the workflow with a coauthor, RA, or colleague.

Habit to keep: before asking an agent to act, name the scope, context, forbidden areas, evidence, and human review.